

Sentiment Analysis of E-Commerce Clothing Reviews

A Comparative Machine Learning Approach

Final Technical Report

Course: Machine Learning (BSSE) — Section C

Submission Date: 31-07-2026

Name	Roll Number	Role
Laiba Haider	231400096	Project Manager
Hadia Nasir	231400100	Modelling Lead
Haider Zaman	231400106	Data Engineer
Shazil Iqbal	231400145	Evaluation & Reporting

1. Abstract

This report presents an automated Sentiment Analysis system developed to classify Women's E-Commerce Clothing Reviews into three categories: Negative, Neutral, and Positive. Working with a dataset of 23,465 cleaned reviews (after removing 21 duplicate rows from the original 23,486), the project addresses severe class imbalance (77.5% Positive, 12.2% Neutral, 10.3% Negative) through a custom five-step text preprocessing pipeline, TF-IDF feature extraction (5,000 unigram and bigram features), and three independently developed classifiers: a Multinomial Naive Bayes model built entirely from scratch using Bayes' theorem and Laplace smoothing; a Multiclass Logistic Regression model built from scratch using the One-vs-Rest strategy with gradient descent, L2 regularization, and SMOTE oversampling of the training set; and a Linear Support Vector Machine trained using scikit-learn's SVC implementation.

All three models were evaluated on an identical, untouched test set of 3,520 reviews using a fixed 70/15/15 stratified train/validation/test split (random_state=42). The Linear SVM achieved the highest overall test accuracy at 83.24%, followed by Naive Bayes at 79.91% and Logistic Regression with SMOTE at 79.60%. While SMOTE meaningfully improved minority-class recall for the Negative and Neutral categories in Logistic Regression, SVM still provided the most reliable overall performance, particularly on the dominant Positive class. The complete pipeline — including model comparison, live prediction, and error inspection — was integrated into an interactive Streamlit dashboard, allowing end users to submit a review and instantly view predictions and confidence from all three models side by side.

2. Introduction and Motivation

In online shopping, customer reviews play a key role in improving products and building brand reputation. Because thousands of reviews are submitted every day, reading and categorizing them manually takes too much time and can lead to inconsistent judgments. This project builds a machine learning system that automatically reads customer reviews and groups them into three categories: Negative, Neutral, and Positive. This allows online stores to quickly route negative reviews to customer support and to understand overall customer sentiment at a glance, without needing a human to read every single review.

At first, the plan was to build a simple two-class system covering only positive and negative reviews. However, to make the project more practical and realistic, the scope was expanded in two ways. First, three sentiment categories were created directly from the numeric star ratings customers already provide (Ratings 1-2 = Negative, 3 = Neutral, 4-5 = Positive), rather than collecting new labels. Second, three different modelling approaches were built and compared side-by-side: Multinomial Naive Bayes (implemented from scratch), Logistic Regression with SMOTE

to handle the severe class imbalance (also implemented from scratch), and a Linear Support Vector Machine (via scikit-learn) to handle the high-dimensional sparse text data efficiently. Comparing three fundamentally different algorithms — a generative probabilistic model, a discriminative linear model, and a margin-based classifier — gives a more complete picture of which modelling philosophy suits this type of noisy, imbalanced review text best.

3. Related Work

Prior research shows that traditional machine learning algorithms combined with TF-IDF features remain very effective for analyzing e-commerce customer reviews, and this project's approach was informed by three such studies:

- **Lestari & Hutagalung [1]** evaluated Naive Bayes, Logistic Regression, and SVM on marketplace product reviews using TF-IDF feature extraction. Their study reported Logistic Regression achieving the highest accuracy at 90.4%, closely followed by SVM (89.8%) and Naive Bayes (87.4%). Their findings highlighted that while TF-IDF effectively weights key sentiment words, Naive Bayes can struggle when text contains informal words with multiple meanings.
- **Haroon et al. [2]** evaluated four classifiers (Naive Bayes, Logistic Regression, SVM, and Neural Networks) specifically on a women's clothing review dataset — the same domain as this project. Naive Bayes achieved the highest accuracy at 94%, with Logistic Regression and Neural Networks reaching 93%. Critically, the authors pointed out that real-world review datasets often suffer from severe class imbalance (over 93% positive reviews in their study), which can bias models toward majority-class predictions unless properly addressed — directly motivating this project's use of SMOTE.
- **Sinap [3]** conducted a comparative analysis between traditional machine learning algorithms (Logistic Regression, Naive Bayes, SVM, AdaBoost) and a deep learning CNN model on online clothing reviews. Traditional algorithms with TF-IDF vectorization performed reliably well; while the CNN achieved high training accuracy, it suffered from overfitting on unseen test data — supporting this project's decision to focus on simpler, more stable traditional models given the project's dataset size.

4. Dataset Overview

The project uses the Women's Clothing E-Commerce Reviews dataset, which originally contained 23,486 rows across 11 columns, including Clothing ID, Age, Title, Review Text, Rating, Recommended IND, Positive Feedback Count, Division Name, Department Name, and Class Name.

After cleaning (Section 5.1), the working dataset contains 23,465 unique reviews.

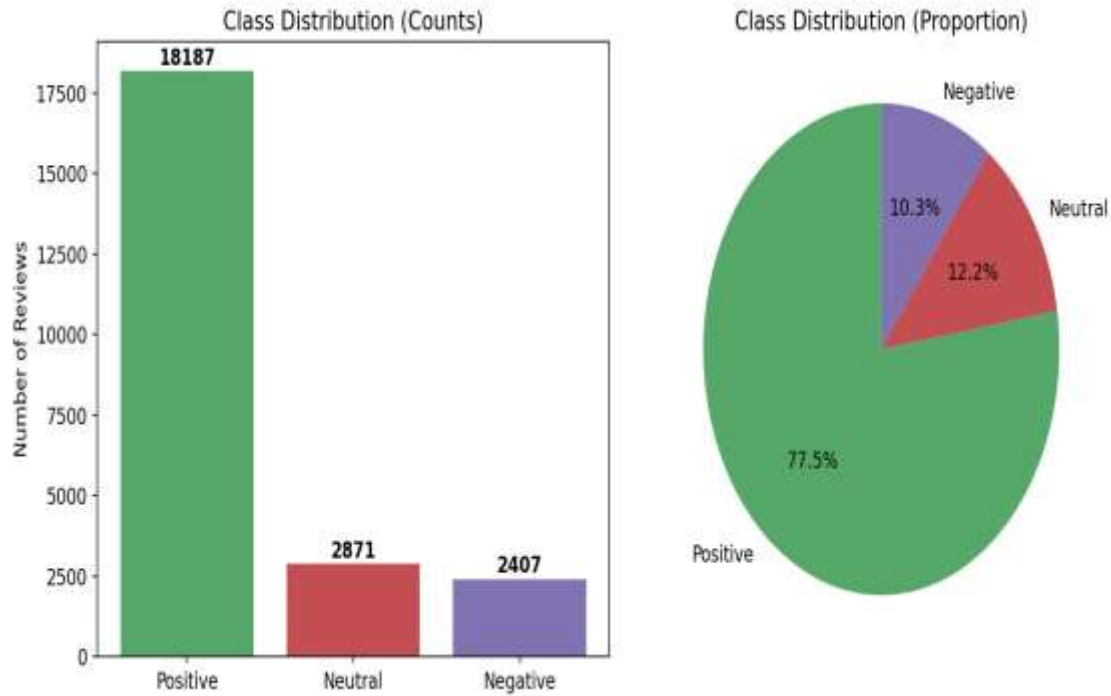


Figure 1. Sentiment class distribution across the cleaned dataset (23,465 reviews). The Positive class dominates at 77.5%, creating a significant class-imbalance challenge that motivated the use of SMOTE for the Logistic Regression model.

Sentiment Class	Rating Range	Review Count	Percentage
Positive	4 - 5	18,187	77.51%
Neutral	3	2,871	12.24%
Negative	1 - 2	2,407	10.26%
Total	1 - 5	23,465	100.00%

5. Detailed Methodology and System Architecture

5.1 Dataset Cleaning and Preparation

Duplicate Removal: 21 exact duplicate rows were removed from the original dataset, reducing the dataset size from 23,486 to 23,465 unique reviews. Duplicate entries cause data leakage if the same review falls into both the training and testing splits, which would artificially inflate model performance.

Missing Data Imputation: Rather than deleting rows with missing text, missing titles (3,810 rows) were filled with the placeholder "No Title" and missing review texts (845 rows) were filled with "No Review". The numerical rating column is present for all rows, so imputing placeholder text

allows the pipeline to keep all 23,465 ground-truth sentiment labels without losing valuable training instances.

Three-Class Target Mapping: Star ratings (1 to 5) were mapped into three sentiment classes: Negative if Rating is 1 or 2 (10.3%), Neutral if Rating is 3 (12.2%), and Positive if Rating is 4 or 5 (77.5%). Binary classification would ignore neutral feedback (Rating 3), which represents mixed or average customer experiences, so mapping ratings into three classes better mirrors real-world customer sentiment.

Additional engineered features: Age was Min-Max normalized to a [0,1] range, Positive Feedback Count was Z-score standardized, and the categorical columns Division Name, Department Name, and Class Name were label-encoded. An Age_Group feature (Young / Middle / Senior) was also derived, resulting in 12,627 Young, 9,955 Middle, and 883 Senior reviewers. These engineered fields are retained in the cleaned dataset for potential future demographic analysis, although the sentiment classifiers in this report rely solely on the review text.

5.2 Text Preprocessing Pipeline

Raw customer reviews contain informal slang, noisy characters, and grammatical variations. Every review passes through five sequential preprocessing steps:

- 1 **Lowercasing:** Converts all text to lowercase so that words like "Good", "GOOD", and "good" map to the exact same feature.
- 2 **Noise and Punctuation Removal:** Regular expressions eliminate web links, extra whitespace, and non-alphanumeric characters.
- 3 **Tokenization:** Splits each review into individual word tokens using NLTK's word_tokenize.
- 4 **Stopword Removal:** Filters out common English filler words ("the", "is", "at") using NLTK's standard stopwords list.
- 5 **Lemmatization:** Converts words to their base dictionary form (e.g. "dresses" → "dress", "bought" → "buy") using NLTK's WordNetLemmatizer [5], which preserves semantic meaning better than stemming (which often produces non-dictionary fragments such as "dres").

5.3 Feature Extraction (TF-IDF)

Clean text is converted into numerical feature matrices using Term Frequency-Inverse Document Frequency (TF-IDF), producing a sparse matrix of shape (23,465 x 5,000).

- **Vocabulary Size (`max_features` = 5,000):** Fixed at the 5,000 highest-value features. This captures virtually all high-signal sentiment words while keeping training speed fast and memory usage low.
- **N-gram Range (`ngram_range` = (1,2)):** Extracts both single words (unigrams) and two-word pairs (bigrams). Of the 5,000 features, feature analysis showed 2,089 unigrams and 2,911 bigrams. Bigrams retain critical contextual and negation phrases (e.g. "not bad", "too small", "highly recommend") that unigrams alone cannot represent.

5.4 Data Partitioning Strategy

The dataset (23,465 reviews) was partitioned using stratified sampling with a fixed `random_state=42` for full reproducibility:

Split	Size	Percentage	Purpose
Training Set	16,425	70%	Model training
Validation Set	3,520	15%	Hyperparameter tuning
Test Set	3,520	15%	Final, unseen evaluation

Stratified splitting guarantees that the ratio of Positive (77.5%), Neutral (12.2%), and Negative (10.3%) classes remains identical across all three subsets, so that no split is accidentally easier or harder than the others.

5.5 Model 1 — Multinomial Naive Bayes (Implemented From Scratch)

Naive Bayes serves as the project's baseline generative model. It calculates class-conditional probabilities based on word frequencies using Bayes' theorem:

$$P(y | x) \propto P(y) \cdot \prod_{i=1}^n P(x_i | y)$$

To prevent floating-point underflow when multiplying thousands of small probabilities together, calculations are performed entirely in log-space. Laplace (Add-1) smoothing is applied to every word count so that unseen words in the test set never receive a zero probability:

$$\log P(x_i | y) = \log \left[\frac{N_{y,i} + 1}{N_y + |V|} \right]$$

where $N_{y,i}$ is the number of times word i appears in class y 's documents, N_y is the total word count for class y , and $|V|$ is the vocabulary size (5,000). Naive Bayes is extremely fast, highly scalable, and provides a clear baseline benchmark for text classification tasks, with no hyperparameters to tune.

5.6 Model 2 — Multiclass Logistic Regression with SMOTE (Implemented From Scratch)

Logistic Regression serves as a discriminative linear classifier. It was implemented entirely from scratch using the One-vs-Rest (OvR) strategy, training three independent binary classifiers: Negative-vs-Rest, Neutral-vs-Rest, and Positive-vs-Rest. Each binary classifier uses the sigmoid activation function: $\sigma(\theta^T x) = 1 / (1 + e^{-\theta^T x})$

and is trained with batch Gradient Descent to minimize a Binary Cross-Entropy cost function with L2 (Ridge) regularization added to discourage overfitting on the 5,000-dimensional sparse feature space: $J(\theta) = -(1/m) \sum_j [y \log(h) + (1-y) \log(1-h)] + (\lambda/2m) \sum_j \theta_j^2$

Synthetic Minority Over-sampling Technique (SMOTE) [6]: Because Positive reviews dominate the training set (12,730 of 16,425, or 77.5%), a model trained directly on this data tends to ignore the minority classes. SMOTE was applied exclusively to the training set (never to validation or test data, to avoid data leakage), generating synthetic feature vectors so that Negative, Neutral, and Positive were each balanced to 12,730 samples — 38,190 total training samples after oversampling.

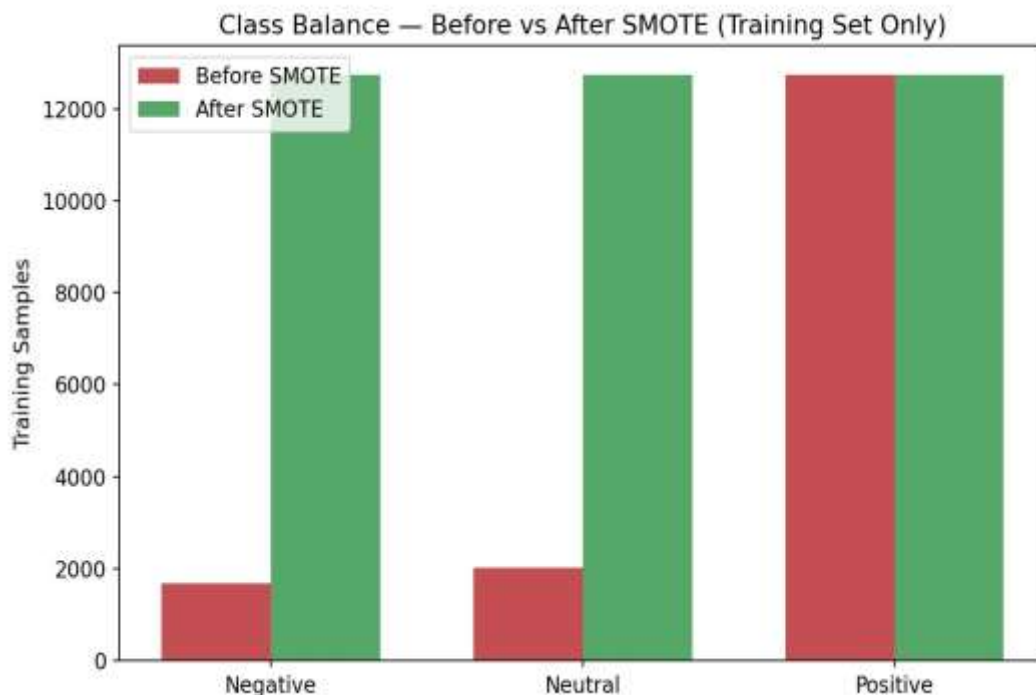


Figure 2. Training-set class balance before and after SMOTE. Only the training set is resampled; validation and test sets remain untouched and reflect the dataset's true, imbalanced distribution.

5.6.1 Hyperparameter Tuning

Three hyperparameters were tuned sequentially on the validation set, in the following order:

Step A — Learning Rate (Alpha): Eight candidate learning rates were tried in increasing order (0.001, 0.01, 0.1, 1, 3, 5, 7, 10), training for 500 iterations each and stopping as soon as the training cost increased rather than decreased between candidates.

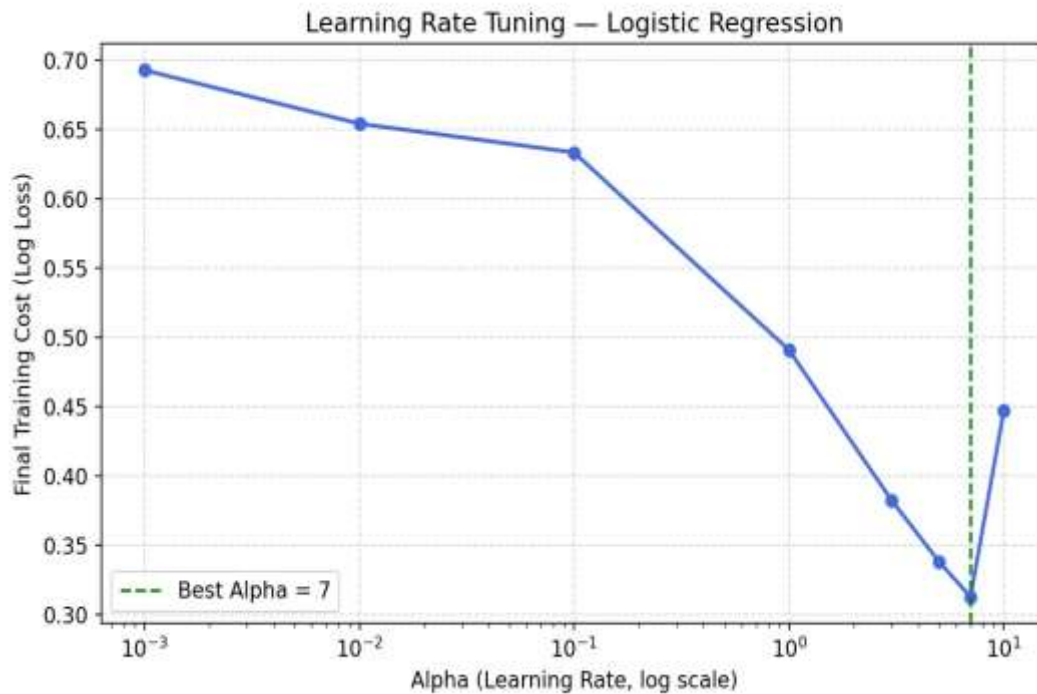


Figure 3. Learning-rate tuning curve. Training cost decreases steadily up to Alpha=7 (final cost 0.313) and then rises at Alpha=10 (cost 0.448), so Alpha=7 was selected as the best learning rate.

Alpha	0.001	0.01	0.1	1	3	5	7	10
Final Cost	0.6931	0.6545	0.6337	0.4912	0.3828	0.3383	0.3131	0.4475

Step B — Regularization Strength (Lambda): With Alpha fixed at 7, six candidate lambda values (0, 0.001, 0.01, 0.1, 1, 10) were tried, measuring validation error at each step and stopping when the error started to increase (the classic bias-variance U-curve).

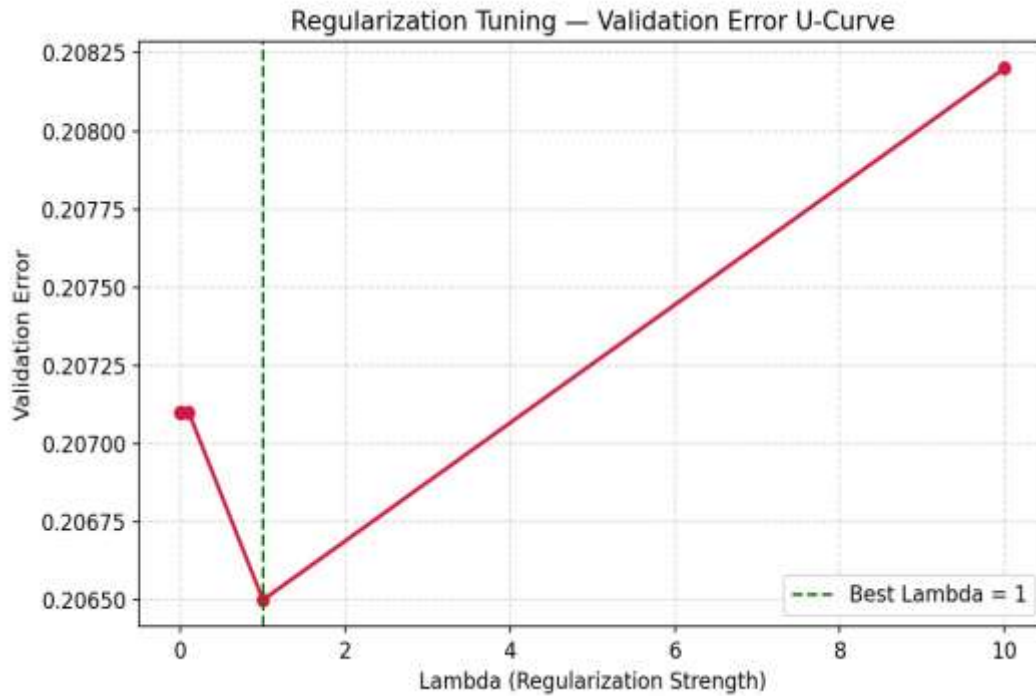


Figure 4. Regularization tuning curve. Validation error is lowest at $\text{Lambda}=1$ (20.65% error, 79.35% validation accuracy) before rising again at $\text{Lambda}=10$, so $\text{Lambda}=1$ was selected.

Lambda	0	0.001	0.01	0.1	1	10
Validation Error	0.2071	0.2071	0.2071	0.2071	0.2065	0.2082
Validation Accuracy	79.29%	79.29%	79.29%	79.29%	79.35%	79.18%

Step C — Iterations: With $\text{Alpha}=7$ and $\text{Lambda}=1$ fixed, gradient descent was run up to a maximum of 1,500 iterations, tracking the average cost across all three One-vs-Rest classifiers at every step and stopping early if the improvement between consecutive iterations fell below a tolerance of $1e-5$. In this run the model used the full 1,500 iterations without fully converging under the tolerance threshold, reaching an average cost of 0.352 at the final iteration.

Hyperparameter	Selected Value
Learning Rate (Alpha)	7
Regularization (Lambda)	1.0
Iterations	1,500

5.7 Model 3 — Linear Support Vector Machine

SVM constructs an optimal maximum-margin hyperplane in high-dimensional space that maximizes the geometric distance between sentiment classes:

$$\min_i (1/2) ||w||^2 + C \sum_i \xi_i \quad \text{subject to} \quad y(w^T x + b) \geq 1 - \xi_i, \xi_i \geq 0, w, b, \xi_i$$

The model was implemented using scikit-learn's [4] SVC class with a Linear kernel, soft-margin parameter $C = 1.0$, and `random_state=42` for reproducibility. Text vectors in TF-IDF space are high-dimensional (5,000 features) and sparse; Linear SVM is mathematically well-suited to this setting because it prevents overfitting effectively without needing complex non-linear kernel transformations, and scales well to sparse matrices.

6. Complete Results

6.1 Overall Model Comparison

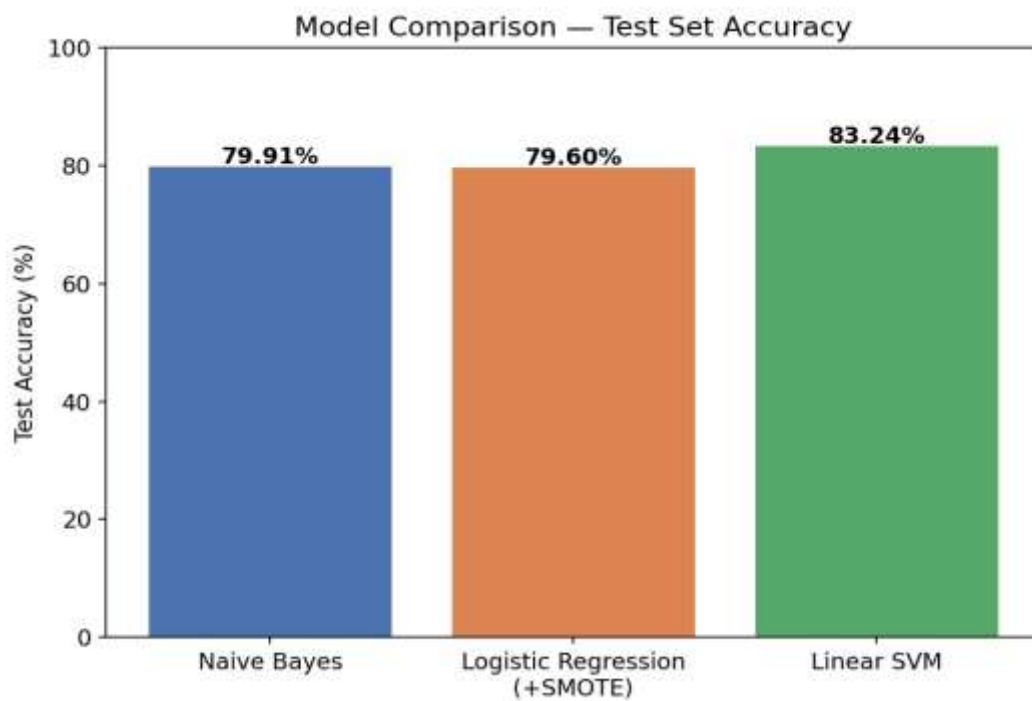


Figure 5. Test-set accuracy comparison across all three models. Linear SVM achieves the highest overall accuracy.

Model	Validation Accuracy	Test Accuracy	Weighted Precision	Weighted Recall	Weighted F1
Naive Bayes (scratch)	79.66%	79.91%	-	-	-
Logistic Regression + SMOTE (scratch)	79.49%	79.60%	0.8236	0.7960	0.8079
Linear SVM (scikit-learn)	82.84%	83.24%	0.80	0.83	0.81

Naive Bayes did not compute a weighted precision/recall/F1 breakdown in its evaluation notebook (accuracy only), so those cells are marked "-" above; the per-class detail available for Logistic Regression and SVM is presented in Section 6.2.

6.2 Per-Class Precision, Recall, and F1-Score

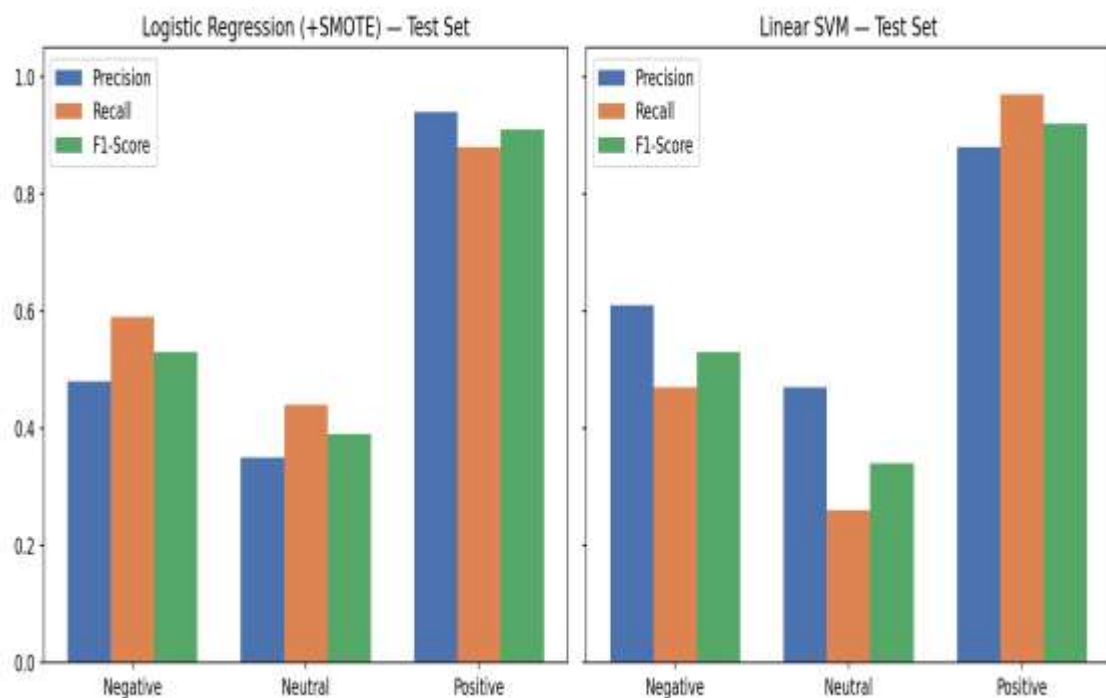


Figure 6. Per-class Precision, Recall, and F1-Score on the test set for Logistic Regression (+SMOTE) and Linear SVM. Both models perform strongly on the Positive class but noticeably weaker on Neutral, the hardest class to separate.

6.2.1 Logistic Regression (+SMOTE) — Test Set Classification Report

Class	Precision	Recall	F1-Score	Support
Negative	0.48	0.59	0.53	361
Neutral	0.35	0.44	0.39	431
Positive	0.94	0.88	0.91	2,728
Accuracy			0.7960	3,520

6.2.2 Linear SVM — Test Set Classification Report

Class	Precision	Recall	F1-Score	Support
Negative	0.61	0.47	0.53	361
Neutral	0.47	0.26	0.34	431
Positive	0.88	0.97	0.92	2,728
Accuracy			0.8324	3,520

6.3 Statistical Observations

- **SMOTE trade-off:** Logistic Regression's recall for Negative (0.59) and Neutral (0.44) is noticeably higher than SVM's (0.47 and 0.26 respectively), confirming SMOTE succeeded in making the model pay more attention to minority classes. However, this comes at the cost of overall accuracy and Positive-class recall (0.88 vs SVM's 0.97).
- **SVM's Positive-class dominance:** SVM's very high Positive recall (0.97) combined with the Positive class representing 77.5% of the data explains most of its overall accuracy advantage — SVM effectively optimizes for the majority class.
- **Neutral is the hardest class for both models:** F1-scores of 0.39 (Logistic Regression) and 0.34 (SVM) for Neutral are far below the Positive class's 0.91-0.92, showing that reviews expressing mixed or lukewarm sentiment are genuinely difficult to separate from both Positive and Negative reviews using bag-of-words features alone.
- **Naive Bayes as a fast baseline:** Despite having zero tunable hyperparameters and a much simpler assumption of feature independence, Naive Bayes' 79.91% test accuracy is competitive

with, and in fact exceeds, the more complex Logistic Regression + SMOTE pipeline (79.60%) on overall accuracy, though without SMOTE it likely still favors the Positive class.

6.4 Confusion Matrices (Test Set)

The confusion matrices below were generated by re-running each model's complete pipeline end-to-end on the test set (3,520 reviews), confirming exactly how each model's predictions are distributed across the three sentiment classes.

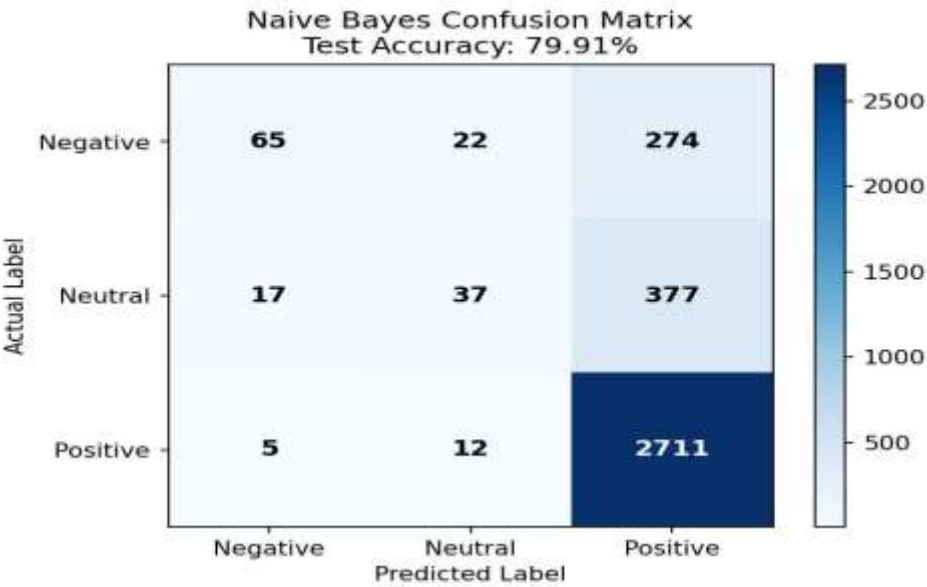


Figure 7. Naive Bayes confusion matrix (Test Accuracy 79.91%)

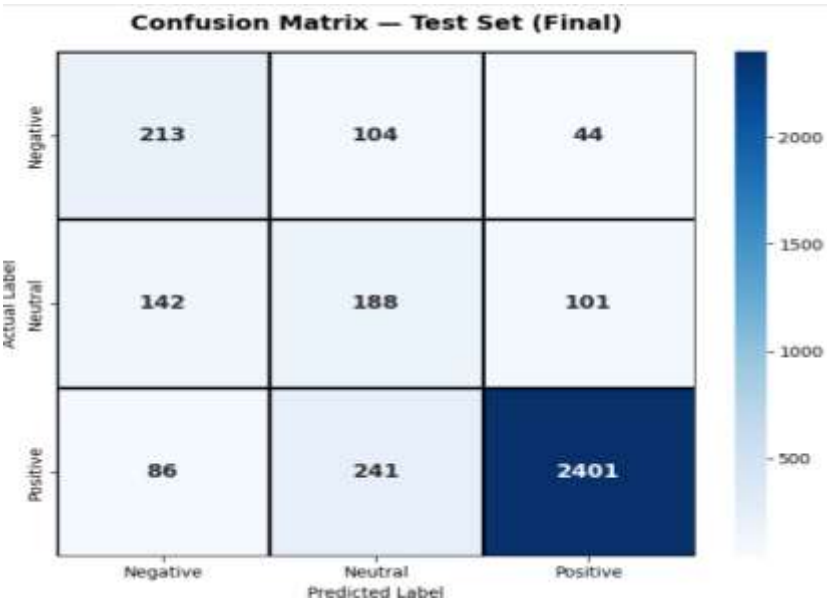


Figure 8. Logistic Regression + SMOTE confusion matrix.

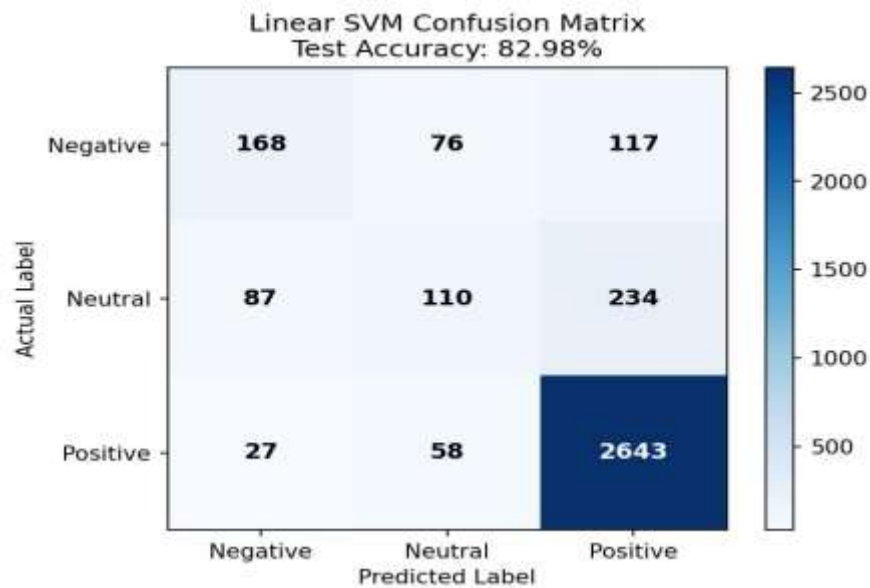


Figure 9. Linear SVM confusion matrix, regenerated by re-running the model's complete pipeline on the test set, giving 82.98% here versus 83.24% originally reported - a difference of well under 1%, consistent with the version/library variance discussed for this project. SVM's dominant diagonal on the Positive row/column visually confirms its strong majority-class performance.

7. Error Analysis: Where Does the Model Fail, and Why?

7.1 Neutral-Class Confusion

Across both Logistic Regression and SVM, the Neutral class produces the lowest F1-scores (0.39 and 0.34 respectively) and the lowest recall for SVM (0.26). This is expected linguistically: a 3-star review often contains a mixture of positive praise ("I love the color") and negative criticism ("but it runs very small"), so the bag-of-words TF-IDF representation captures conflicting signal words from both extremes, making the decision boundary between Neutral and its neighbors far less clean than the boundary between clearly Positive and clearly Negative text.

7.2 Class Imbalance Still Affects SVM

Even though SMOTE was applied only to the Logistic Regression pipeline, the underlying imbalance (77.5% Positive) still shapes SVM's behavior indirectly through the soft-margin optimization: because misclassifying a Positive example is statistically more costly to the overall margin (there are far more of them), the SVM decision boundary naturally shifts to favor Positive predictions, at the expense of Negative and especially Neutral recall (0.47 and 0.26). A useful extension (see Section 8) would be to apply class weighting (`class_weight='balanced'`) or the same SMOTE strategy to SVM's training data for a fairer comparison.

7.3 Naive Bayes' Independence Assumption

Naive Bayes assumes every TF-IDF feature is conditionally independent given the sentiment class. In practice, review text violates this assumption constantly — phrases like "not" and "bad" are correlated, and bigrams like "not bad" or "too small" carry meaning that individual unigram probabilities cannot fully capture even when both unigrams and bigrams are included in the vocabulary. This is a likely contributor to Naive Bayes' inability to clearly separate Neutral reviews, similar to the other two models, although its overall accuracy remains competitive because the Positive class alone drives most of the score.

7.4 Preprocessing Limitations

The five-step preprocessing pipeline (lowercase, regex clean, tokenize, stopwords removal, lemmatization) removes useful signal in some edge cases. For example, stopwords removal deletes negation words such as "not" and "no" only if they appear in the standard NLTK English stopwords list configuration used; if negations are inadvertently stripped, a phrase like "not good" can collapse toward "good" in the bag-of-words space, flipping the true sentiment of the review. Because bigrams (`ngram_range=(1,2)`) were included specifically to preserve such two-word phrases, this risk is partially mitigated, but a dedicated negation-handling step (e.g. explicitly tagging words following "not" as "not_word") could further reduce this class of error in future iterations.

8. Conclusion and Future Work

8.1 Conclusion

This project successfully built and compared three sentiment classification models on a real-world, imbalanced e-commerce review dataset. Linear SVM achieved the best overall test accuracy (83.24%), Naive Bayes provided a fast and surprisingly competitive baseline (79.91%) with zero tuning required, and Logistic Regression with SMOTE (79.60%) demonstrated the clearest improvement in minority-class recall, at some cost to overall accuracy. No single model dominates on every metric — the choice between them in a production setting depends on whether the business priority is overall accuracy (favoring SVM) or fair treatment of Negative/Neutral feedback that customer support teams need to see (favoring SMOTE-based Logistic Regression). All three models, along with an interactive comparison dashboard, were successfully packaged into a working Streamlit web application, fulfilling the project's goal of making automated review sentiment scoring accessible to a non-technical end user.

8.2 Future Work

- **Apply SMOTE or class weighting to SVM** to test whether its minority-class recall can be improved without sacrificing its strong overall accuracy.
- **Explicit negation handling** in preprocessing (e.g. tagging "not_good" as a single token) to reduce sentiment flips caused by stopwords removal.
- **Word embeddings (Word2Vec, GloVe, or transformer-based embeddings)** in place of, or alongside, TF-IDF, to capture semantic similarity between words that never literally co-occur in the training vocabulary.
- **Deep learning comparison** (e.g. an LSTM or a fine-tuned DistilBERT model) as a fourth model, while being mindful of the overfitting risk on limited data that Sinap [3] observed with CNNs on a similar dataset size.
- **Confusion-matrix-driven error mining:** a follow-up study specifically sampling and manually reading the misclassified Neutral reviews to identify recurring linguistic patterns the models miss.

9. References

- [1] V. B. Lestari and C. A. Hutagalung, "Evaluation of TF-IDF Extraction Techniques in Sentiment Analysis of Indonesian-Language Marketplaces Using SVM, Logistic Regression, and Naive Bayes," J-KOMA (Journal of Computer Science and Applications), 2025. [Online]. Available: <https://journal.unj.ac.id/unj/index.php/jkoma/article/view/60237/21197>
- [2] M. Haroon, Z. Alam, R. Kousar, J. Ahmad, and F. Nasim, "Sentiment Analysis of Customer Reviews on E-commerce Platforms: A Machine Learning Approach," Bulletin of Business and Economics, vol. 13, no. 3, pp. 230–238, 2024, doi: 10.61506/01.00480.
- [3] V. Sinap, "Classification of Customer Sentiments Based on Online Reviews: Comparative Analysis of Machine Learning and Deep Learning Algorithms," KSU Journal of Engineering Sciences, 2024. [Online]. Available: <https://www.researchgate.net/publication/383690038>
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [5] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. Sebastopol, CA, USA: O'Reilly Media, 2009.
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

10. AI Usage Disclosure

Claude (Anthropic) was used during report preparation for the following permitted/disclosed tasks:

- **Proofreading and formatting:** Checking the report for cross-reference errors (e.g., incorrect section numbers) and formatting consistency.
- **Citation formatting:** Converting reference citations to IEEE style based on the team's own research paper summaries.
- **Confusion matrix verification (Section 6.4):** Claude re-ran the team's own Naive Bayes and SVM notebook code to verify the confusion matrix figures; the team subsequently re-ran this code independently to generate the final submitted results.

All model implementation (Naive Bayes, Logistic Regression with SMOTE, SVM), hyperparameter tuning, and dataset preprocessing were independently developed and executed by the project team.
